Sovereign AI

Testing & Verification Guide — Mistral 7B BFSI Fine-Tuning on Colab

Phase-by-Phase Checks, Expected Output, and Failure Signs

Companion to the Mistral7B_Colab_FineTuning_Roadmap and the ready-to-run Colab notebook

Prepared for: Sovereign AI project
Date: 23 September 2026

General Debugging Rule of Thumb

If a phase's test fails, don't skip ahead — every later phase assumes the previous one produced a valid artifact (tokenizer, adapter, merged model, GGUF file), and failures compound silently. For example, a bad Phase 4 tokenizer will still let Phase 7 training "succeed", but the model will generate subtly wrong tokens in Phase 8 that trace back to a step you already passed.

1 Environment Setup

Test GPU is attached

```
!nvidia-smi
```

Pass: shows a GPU (T4 or A100) with memory stats.

Fail / watch for: "command not found" — go to Runtime > Change runtime type > GPU.

Test library versions and CUDA visibility

```
import torch, transformers, peft, bitsandbytes, trl
print("torch:", torch.__version__, "| CUDA available:", torch.cuda.is_available())
print("transformers:", transformers.__version__)
print("GPU name:", torch.cuda.get_device_name(0))
```

Pass: "CUDA available: True" and a GPU name prints.

Fail / watch for: False means the runtime isn't GPU-backed or drivers didn't load — restart runtime.

Test HF login and Drive mount

```
from huggingface_hub import whoami
print(whoami()) # should print your HF username, not an error

import os
print(os.path.exists(WORKDIR), WORKDIR) # should print True
```

2 Data Collection

Test the file was written correctly

```
import json
with open(DATA_FILE) as f:
lines = f.readlines()
print("Row count:", len(lines))
print("First row:", json.loads(lines[0]))
```

Pass: row count matches expectations; first row parses as valid JSON with instruction/response keys.

Test the HF dataset loaded matches the file

```
print(len(ds)) # should equal the line count above
print(ds.column_names) # should be ['instruction', 'response']
```

Pass: None

Fail / watch for: load_dataset can silently load 0 rows if DATA_FILE path is wrong — check len(ds) > 0 explicitly before continuing.

3 Data Cleaning & Formatting

Visually inspect a few formatted examples

```
for i in range(min(3, len(ds["train"]))):
print(ds["train"][i]["text"])
print("-" * 40)
```

Pass: each example starts with <s>[INST] and ends with </s>, with your actual instruction/response text in between — no None, no double-encoded quotes.

Test the split sizes

```
print("train:", len(ds["train"]), "test:", len(ds["test"]))
```

Pass: both are non-zero.

Fail / watch for: with a tiny sample dataset, test_size=0.2 on 5 rows may round to 1 — fine for a pipeline smoke test, but before a real run, make sure test has 10-20+ examples so Phase 8 evaluation is meaningful.

4 Tokenizer Extension

Test the vocab actually grew

```
print("Before:", 32000, "| After:", len(tokenizer))
assert len(tokenizer) == 32000 + len(new_tokens)
```

Test the new BFSI terms now tokenize more efficiently

```
for t in bfsi_terms:
print(f"{t:12s} -> {tokenizer.tokenize(t)}")
```

Pass: terms you added now show up as a single token (e.g. ['■NPA'] instead of ['■N', 'P', 'A']).

Fail / watch for: terms that were already fine (not added) look the same as before — that's expected, not a bug.

Test round-trip encode/decode doesn't corrupt text

```
sample = "The NPA ratio and CRR requirements under IRDAI guidelines."
ids = tokenizer.encode(sample)
decoded = tokenizer.decode(ids)
print(decoded)
assert "NPA" in decoded and "CRR" in decoded
```

5 Load Model in 4-bit

Test the model loaded and check memory footprint

```
print(model.config.vocab_size, "vs tokenizer:", len(tokenizer)) # should match if resized

print(f"GPU memory allocated: {torch.cuda.memory_allocated()/1e9:.2f} GB")
print(f"GPU memory reserved: {torch.cuda.memory_reserved()/1e9:.2f} GB")
```

Pass: on T4 (16GB): allocated should be roughly 4-5GB for the 4-bit base model, leaving headroom for training activations.

Fail / watch for: if it's already close to 16GB here, you'll OOM in Phase 7 — restart runtime and check nothing else is holding GPU memory (!nvidia-smi).

Test a forward pass runs without error (before attaching LoRA)

```
test_input = tokenizer("Hello, this is a test.", return_tensors="pt").to(model.device)
with torch.no_grad():
out = model(**test_input)
print(out.logits.shape) # (1, seq_len, vocab_size)
```

6 LoRA Configuration

Test only a small fraction of params are trainable — the whole point of LoRA

```
model.print_trainable_parameters()
```

Pass: something like "trainable params: 4,194,304 || all params: 7,245,000,000 || trainable%: 0.058".

Fail / watch for: if trainable% is near 100%, LoRA wasn't applied correctly — check target_modules matches actual layer names (see next cell).

```
for name, _ in model.named_modules():
if "q_proj" in name:
print(name)
break
```

7 Fine-Tuning

Test the loss is actually decreasing — the single most important check

Watch the logging_steps=5 output during trainer.train(): loss should trend down over the first 20-30 steps, not stay flat or spike to nan.

Pass: None

Fail / watch for: loss flat/not moving → learning rate too low, or target_modules misconfigured. Loss → nan → learning rate too high, or bf16/fp16 mismatch (use fp16=True on T4, not bf16=True).

Test checkpoints are actually being written to Drive

```
!ls -la {WORKDIR}/checkpoints
```

Pass: checkpoint-N folders appear after each epoch, containing adapter_model.safetensors.

Test you can resume from a checkpoint (simulates surviving a disconnect)

```
trainer.train(resume_from_checkpoint=True)
```

Run this after an intentional interrupt to confirm it picks up rather than restarting from step 0.

Test the saved adapter loads back cleanly

```
from peft import PeftConfig
config = PeftConfig.from_pretrained(ADAPTER_DIR)
print(config.base_model_name_or_path) # should print "mistralai/Mistral-7B-v0.1"
```

8 Evaluation

Before/after comparison — base vs. fine-tuned on the same prompt

```
from peft import PeftModel

# Fine-tuned answer (current `model` already has the adapter attached)
print("FINE-TUNED:", ask("What does CRR stand for?"))

# Disable adapter temporarily to see the base model's answer
with model.disable_adapter():
print("BASE:", ask("What does CRR stand for?"))
```

Pass: the fine-tuned answer is more precise/BFSI-specific than the base answer, and uses terminology consistent with your training examples.

Test for regressions on general (non-BFSI) capability

```
print(ask("What is 12 + 7?"))
print(ask("Write one sentence describing the color blue."))
```

Pass: coherent, sensible answers.

Fail / watch for: if these degrade badly, you likely overfit (too many epochs on too little data) or used too high a learning rate.

Quantify it, don't just eyeball it

```
correct = 0
for ex in ds["test"]:
answer = ask(ex["instruction"])
print(ex["instruction"], "\n->", answer, "\n")
# manually mark correct += 1 after review, or use an LLM-judge call here
```

9 Merge Adapter

Test the merge didn't silently fail

```
inputs = tokenizer("<s>[INST] What does CRR stand for? [/INST]",
return_tensors="pt").to(merged.device)
out = merged.generate(**inputs, max_new_tokens=100)
print(tokenizer.decode(out[0], skip_special_tokens=True))
```

Pass: near-identical output to what Phase 8 produced with the adapter attached.

Fail / watch for: if it reverts to generic (non-fine-tuned) behavior, the merge used the wrong adapter path or base model.

Test file sizes look right

```
!du -sh {MERGED_DIR}
```

Pass: roughly 13-14GB for a merged fp16 7B model.

Fail / watch for: a folder a fraction of that size means saving was incomplete.

10 GGUF Conversion

Test the GGUF file was created and is a plausible size

```
!ls -lh {WORKDIR}/*.gguf
```

Pass: the f16 GGUF is ~13-14GB; the Q4_K_M quantized version should shrink to ~4-4.5GB.

Test the GGUF actually loads and generates, right inside Colab, before downloading it

```
!./llama.cpp/build/bin/llama-cli -m "{WORKDIR}/mistral-bfsi-Q4_K_M.gguf" \
-p "[INST] What does KYC stand for? [/INST]" -n 100
```

Pass: coherent output resembling your fine-tuned model's answers.

Fail / watch for: garbage tokens usually mean the conversion/quantization used the wrong tokenizer/vocab size — re-check that MERGED_DIR actually contains tokenizer.model and tokenizer_config.json.

11 Ollama Deployment (local machine)

Test Ollama sees the model

```
ollama list
```

Pass: mistral-bfsi appears in the list.

Test generation, compare against the Colab GGUF test above

```
ollama run mistral-bfsi "What does KYC stand for?"
```

Test the API endpoint directly (useful once wired into an app)

```
curl http://localhost:11434/api/generate -d '{
"model": "mistral-bfsi",
"prompt": "What does CRR stand for?",
"stream": false
}'
```

Pass: JSON response with a "response" field containing a sensible answer.

12 Monitoring

Test your logging actually captures what you need before relying on it: send a few test prompts through your app/API and confirm each shows up in your log file/sheet with prompt, response, and timestamp. This check matters because if logging is silently broken, you won't have data to build your next fine-tuning round from.